

Explainable Document Q&A; System - Sample Report

1. Executive Summary

Retrieval-Augmented Generation (RAG) combines information retrieval with large language models (LLMs) to answer questions based on ground-truth documents. This application, 'Explainable Document Q&A;', is built using Python 3.11, Streamlit, LangChain, ChromaDB, sentence-transformers, and the Groq LLM API.

2. Technical Specifications & Stack

User Interface: Streamlit provides an interactive chat interface and document upload panel.

Orchestration: LangChain manages document loading, text splitting, and retrieval pipelines.

Vector Store: ChromaDB runs locally in-memory, storing dense vector representations.

Embeddings: The model 'sentence-transformers/all-MiniLM-L6-v2' generates 384-dimensional embeddings.

LLM Provider: Groq API powers the 'llama-3.3-70b-versatile' model for fast, context-bounded generation.

3. Ingestion and Processing Pipeline

Documents are parsed page-by-page using pypdf. Text is chunked with LangChain's RecursiveCharacterTextSplitter configured with a chunk_size of 600 characters and a chunk_overlap of 100 characters. Each chunk preserves metadata including chunk_id, page_number, and source_text.

4. Explainability Layer & Confidence Scoring

A key differentiator of this system is the Explainability Layer. For every user query, the vector store retrieves the top-5 chunks via cosine similarity.

Confidence is assigned based on the top chunk similarity score:

- **High Confidence:** Similarity score above 0.5 (Green badge).
- **Medium Confidence:** Similarity score between 0.3 and 0.5 (Yellow badge).
- **Low Confidence:** Similarity score below 0.3 (Red badge — answer may be unreliable).

5. Security, Environment, and Constraints

API keys are strictly loaded from environment variables (GROQ_API_KEY) and are never hardcoded. If a scanned PDF without extractable text is uploaded, the ingestion pipeline handles it gracefully by throwing an explicit user error.